

Fingerprint-based Smart Door Lock System

Setup and Configurations

Aayush Dhakal

Bachelors in Computer Networking and IT Security

Islington College, Nepal

May 2026

Table of Contents

1	Docker environment setup.....	1
2	Node-RED setup	4
3	InfluxDB setup	6
4	Grafana setup	7
5	Mosquitto setup.....	10

1 Docker environment setup

A directory 'deployment' was created where Mosquitto, Node-RED, InfluxDB and Grafana were setup.

1. A file docker-compose.yaml was created where the following lines were added:

```
version: '3.8'

services:

  nodered:

    image: nodered/node-red:latest

    container_name: node_red

    ports:

      - "1880:1880"

    volumes:

      - ./node-red-data:/data

    depends_on:

      - mosquitto

  influxdb:

    image: influxdb:2.7

    container_name: influxdb

    ports:

      - "8086:8086"

    volumes:
```

```
- ./influxdb-data:/var/lib/influxdb2
```

grafana:

```
image: grafana/grafana:latest
```

```
container_name: grafana
```

ports:

```
- "3000:3000"
```

volumes:

```
- ./grafana-data:/var/lib/grafana
```

depends_on:

```
- influxdb
```

mosquitto:

```
image: eclipse-mosquitto
```

```
container_name: mosquitto
```

volumes:

```
- ./mosquitto/config:/mosquitto/config
```

```
- ./mosquitto/data:/mosquitto/data
```

```
- ./mosquitto/log:/mosquitto/log
```

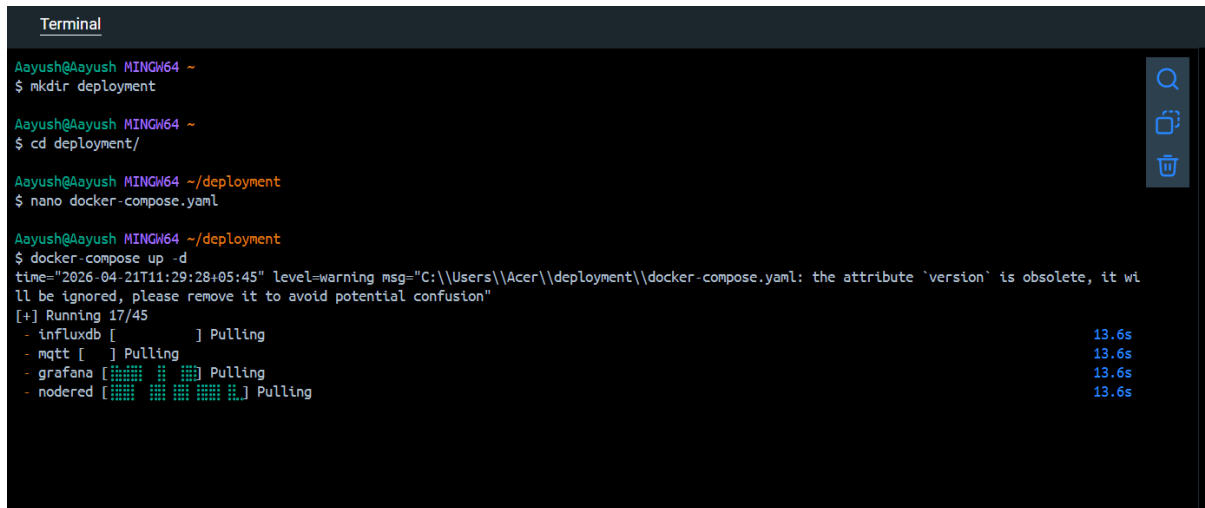
ports:

```
- 1883:1883
```

```
- 9001:9001
```

2. Following command was used to run all four services in the background:

```
docker-compose up -d
```

A terminal window titled "Terminal" showing a series of commands and their outputs. The user is at a prompt "Aayush@Aayush MINGW64 ~". They run "\$ mkdir deployment", "\$ cd deployment/", and "\$ nano docker-compose.yaml". Then they run "\$ docker-compose up -d". The output shows a warning about the 'version' attribute in the docker-compose.yaml file and then a progress bar for pulling four services: influxdb, mqtt, grafana, and nodered. Each service is shown with a progress bar and a time of 13.6s.

```
Terminal
Aayush@Aayush MINGW64 ~
$ mkdir deployment

Aayush@Aayush MINGW64 ~
$ cd deployment/

Aayush@Aayush MINGW64 ~/deployment
$ nano docker-compose.yaml

Aayush@Aayush MINGW64 ~/deployment
$ docker-compose up -d
time="2026-04-21T11:29:28+05:45" level=warning msg="C:\\Users\\Acer\\deployment\\docker-compose.yaml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] Running 17/45
- influxdb [ ] Pulling 13.6s
- mqtt [ ] Pulling 13.6s
- grafana [ ] Pulling 13.6s
- nodered [ ] Pulling 13.6s
```

Figure 1 Docker setup

2 Node-RED setup

Following nodes with the given information were added:

1. MQTT-IN node:
 - a. Host: 192.168.34.195
 - b. Port: 1883
 - c. Topic: home/security/access
 - d. Output: String
2. JSON Node
 - a. Action: Always convert to JavaScript Object.
 - b. Function Node: It contains the following code:

```
// This prepares the data for InfluxDB

msg.payload = {

    status: msg.payload.status,

    user: msg.payload.user

};

// Optional: add a tag to identify the device

msg.tags = {

    location: "Main_Entrance"

};

return msg;
```

- c. Influxdb out node:
- d. Version: 2.0
- e. Server: http://192.168.34.195:8086 (Use your laptop IP).
- f. Token: <InfluxDB API token>
- g. Organization: Islington_IoT (or your chosen name).

- h. Bucket: security_logs.
 - i. Measurement: access_events.
3. Debug node: A debug node was connected to MQTT in to see debug messages.

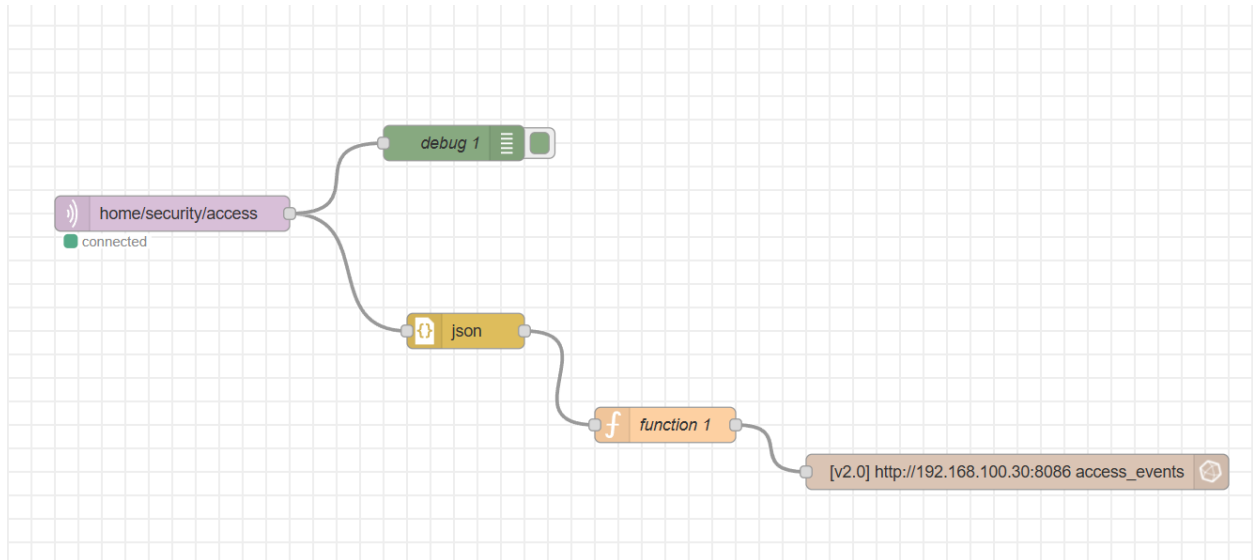


Figure 2 Node-RED setup

3 InfluxDB setup

1. Username and password were set up.
2. A new bucket was created, and the bucket name was given security_logs.
3. All access API token was generated.

4 Grafana setup

1. A new data source was added (InfluxDB).
2. InfluxDB information such as database name, username, password and the InfluxDB URL was entered.
3. Settings was configured:
 - a. Language: Flux
 - b. Organization: Islington_IoT
 - c. Default bucket: security_logs
4. A new visualization was created providing the following values:
 - a. Data source: InfluxDB
 - b. Visualization: table
 - c. Query box:

```
from(bucket: "security_logs")

|> range(start: v.timeRangeStart, stop: v.timeRangeStop)

|> filter(fn: (r) => r["_measurement"] == "access_events")

|> keep(columns: ["_time", "status", "user"])
```

5. Another visualization was created:
 - a. Visualization: Pie chart
 - b. Style: donut
 - c. Labels: Name and value
 - d. Query box:

```
from(bucket: "security_logs")

|> range(start: v.timeRangeStart, stop: v.timeRangeStop)

|> filter(fn: (r) => r["_measurement"] == "access_events")

|> filter(fn: (r) => r["_field"] == "status")

|> group(columns: ["_value"])
```

```
|> count(column: "_field")  
  
|> group()
```

e. In the Value mappings (right sidebar), you can set "Authorized" to Green and "Denied" to Red.

6. Another visualization was created:

a. Visualization: Time series

b. Query box:

```
from(bucket: "security_logs")  
  
|> range(start: v.timeRangeStart, stop: v.timeRangeStop)  
  
|> filter(fn: (r) => r["_measurement"] == "access_events")  
  
|> filter(fn: (r) => r["_field"] == "status")  
  
|> aggregateWindow(every: 1m, fn: count, createEmpty: false)  
  
|> yield(name: "Access Frequency")
```

7. A dashboard was created and all visualizations were added.

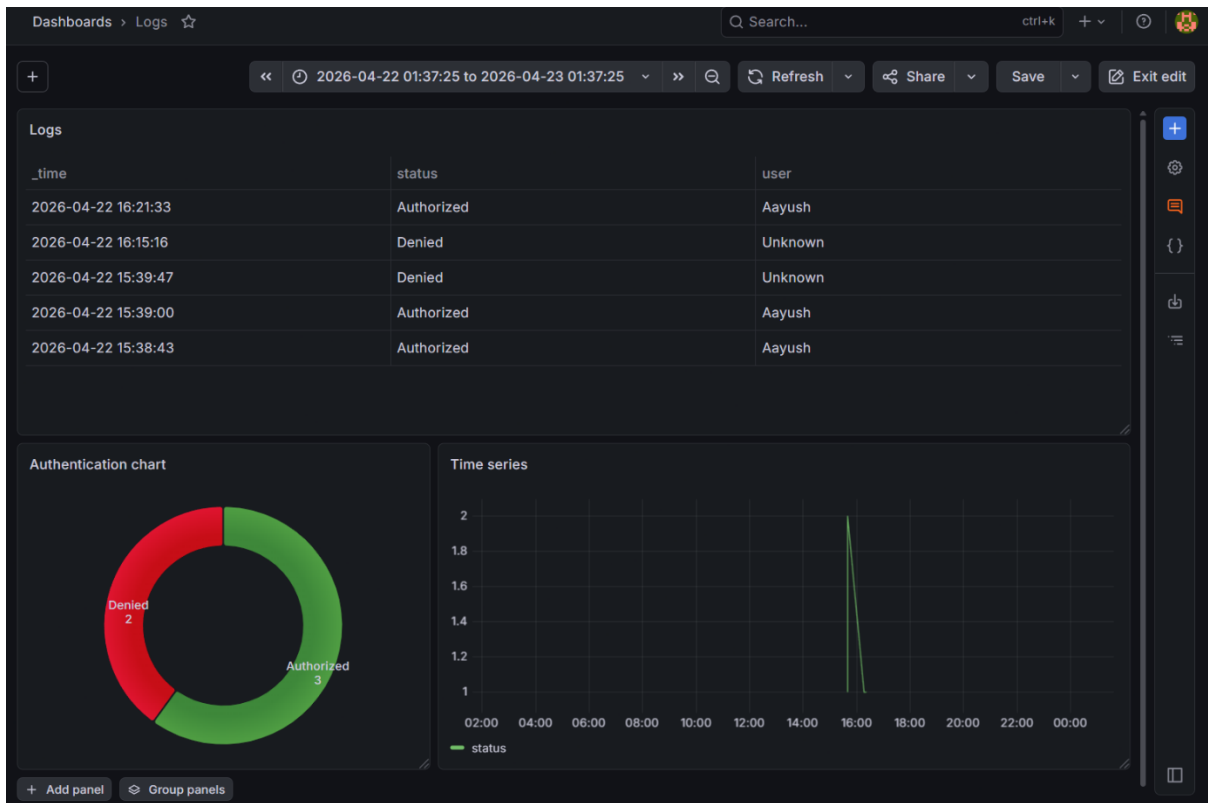


Figure 3 Grafana setup

5 Mosquitto setup

Inside mosquitto/config folder, a mosquitto.conf file was created and following lines were added:

```
persistence true  
  
persistence_location /mosquitto/data/  
  
log_dest file /mosquitto/log/mosquitto.log  
  
  
listener 1883 0.0.0.0  
  
allow_anonymous true
```